

Predictive Maintenance System

Complete Technical Documentation

How the System Works — End to End

Tech Stack: React.js | FastAPI | XGBoost | SHAP | Gemini 2.5 Flash | SQLite

1. System Overview

This system is an AI-powered industrial predictive maintenance platform. It monitors machine sensor readings and predicts two things: (1) whether a machine is likely to fail, and (2) how many operational cycles remain before it needs maintenance. The entire flow — from user input to AI-generated explanation — is handled automatically without any manual data processing steps.

1.1 High-Level Tech Stack

Layer	Technology	Responsibility
Frontend UI	React.js + Tailwind CSS	Dashboard, prediction forms, charts, SHAP visualisation
HTTP Communication	Axios (api.js)	Sends POST/GET requests from React to FastAPI
Backend API	FastAPI (Python)	Receives requests, calls ML models, stores results
ML Inference	XGBoost + SHAP	Runs predictions and computes feature importance
AI Explanations	Google Gemini 2.5 Flash	Generates plain-language explanation + action items
Data Persistence	SQLite + SQLAlchemy	Stores every prediction with timestamp and results
Model Files	pickle (.pkl)	Serialised trained XGBoost models loaded at startup

2. Project Structure

The project is split into two top-level directories: backend (Python/FastAPI) and frontend (React). Below is the complete file tree with the role of each file.

2.1 Backend Structure

File / Folder	Purpose
backend/main.py	FastAPI app entry point. Registers CORS middleware, includes routers, and calls init_db() on startup.
backend/schemas.py	Pydantic models for request/response validation (FailureInput, RULInput, PredictionResponse, HistoryItem).
backend/ml_utils.py	Loads .pkl models at startup. Exposes predict_failure() and predict_rul() functions with SHAP computation.
backend/ai_explain.py	Calls Gemini 2.5 Flash to generate 3-sentence explanation + 3 actions. Has fallback and low-risk static messages.
backend/database.py	SQLite setup. Functions: init_db(), insert_prediction(), get_history(). Timestamps stored in IST.
backend/config.py	Pydantic Settings — reads .env for API keys and model paths.
backend/routers/predict.py	POST /predict/failure and POST /predict/rul endpoints. Calls ml_utils → ai_explain → database.
backend/routers/history.py	GET /history endpoint with optional risk_level and prediction_type filters.

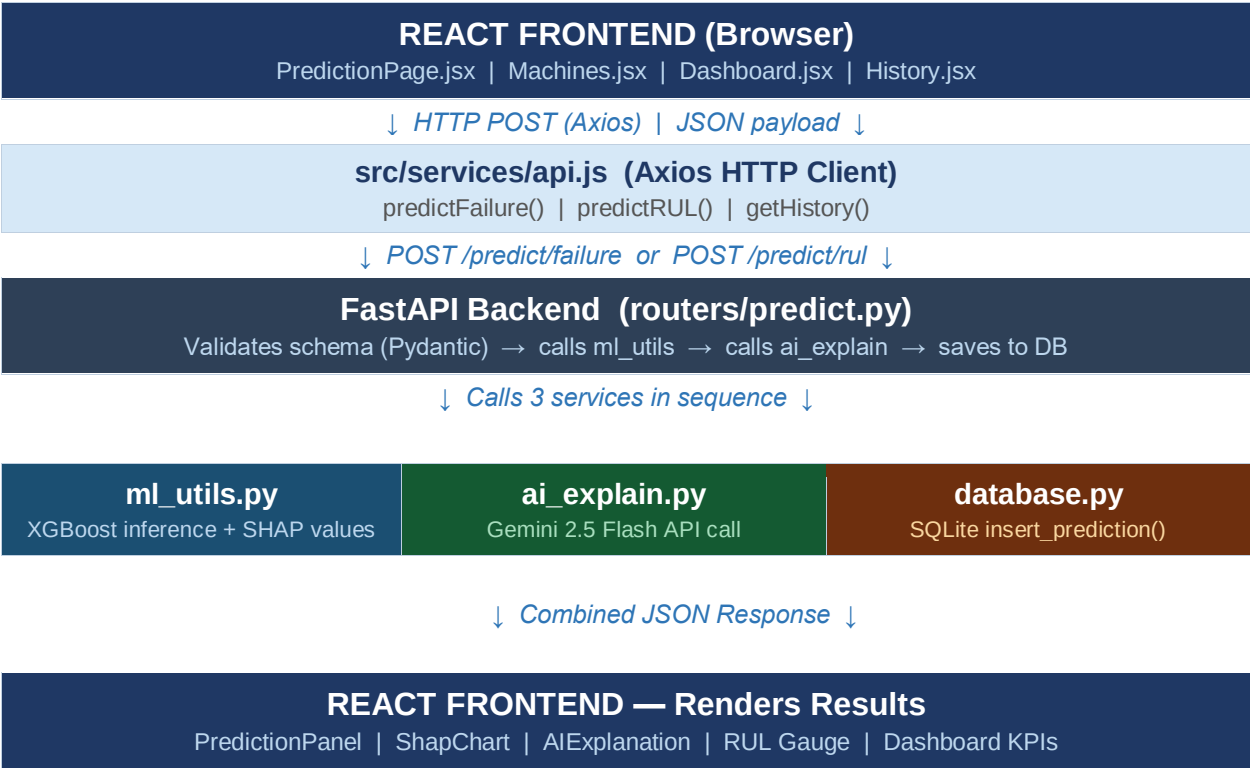
File / Folder	Purpose
backend/ml/train_failure.py	Trains XGBoost classifier on AI4I 2020 CSV. Saves failure_model.pkl.
backend/ml/train_rul.py	Trains XGBoost regressor on CMAPSS FD001 TXT. Saves rul_model.pkl and rul_scaler.pkl.
backend/data/ai4i2020.csv	AI4I 2020 Predictive Maintenance Dataset (10,000 rows, 5 features + failure label).
backend/data/train_FD001.txt	CMAPSS FD001 NASA turbofan engine degradation dataset (21 sensor columns).
backend/models/*.pkl	Serialised trained models: failure_model.pkl, rul_model.pkl, rul_scaler.pkl.

2.2 Frontend Structure

File / Folder	Purpose
src/services/api.js	Axios instance with baseURL. Exports predictFailure(), predictRUL(), getHistory().
src/pages/Prediction/PredictionPage.jsx	Main failure prediction page. Renders Form, PredictionPanel, ShapChart, AIExplanation in a 2-column grid.
src/pages/Prediction/Form.jsx	5-field input form (air_temperature, process_temperature, rotational_speed, torque, tool_wear). Calls onSubmit on submit.
src/pages/Prediction/PredictionPanel.jsx	Shows failure result, confidence score, and risk badge after prediction.
src/pages/Prediction/ShapChart.jsx	Recharts horizontal BarChart. Red bars = increases risk, Cyan bars = reduces risk.
src/pages/Prediction/AIExplanation.jsx	Parses and displays Gemini AI explanation text — splits on 'ACTIONS' keyword.
src/pages/Machines/Machines.jsx	Engine unit selector page. Sends RUL prediction, shows RUL gauge circle, SHAP chart, sensor table.
src/pages/Dashboard/Dashboard.jsx	Overview page. Shows 4 KPI cards, failure risk trend chart, health pie chart, recent predictions table.
src/pages/History/History.jsx	Full prediction history with filters by type and risk level.
src/data/Sensor.js	Static dummy sensor readings for each CMAPSS engine unit. Also exports MACHINE_UNITS and UNIT_RISK_HINT.
src/components/Navbar.jsx	Top navigation bar with page links.
src/animations/pageTransition.js	Framer Motion variants for page enter/exit transitions.
src/Layout.jsx	Wraps all pages with Navbar and AnimatePresence for route transitions.

3. System Architecture

The diagram below shows how data flows through every layer of the system — from the user entering values in the browser, through the FastAPI backend, ML inference, AI explanation generation, database storage, and finally back to the React frontend for display.



4. Frontend–Backend Interaction (Complete Flow)

This section explains exactly how data moves from the user interface through every function call, transformation, and database operation, and returns to the screen.

4.1 Failure Prediction Flow (Predict Page)

Step 1	User fills the 5-field form in Form.jsx: air_temperature, process_temperature, rotational_speed, torque, tool_wear. Clicks 'Run Prediction'.
Step 2	Form.jsx calls onSubmit(formData) prop — which is handleSubmit() in PredictionPage.jsx. State: loading=true, error=null.
Step 3	handleSubmit() calls predictFailure(formData) from api.js. Axios sends POST /predict/failure with JSON body.
Step 4	FastAPI receives request. Pydantic validates FailureInput schema — checks all fields gt:0, types correct. Returns 422 if invalid.
Step 5	predict_failure(data) in ml_utils.py: renames keys to match training columns, creates DataFrame, calls failure_model.predict_proba(). Returns probability.
Step 6	SHAP: failure_explainer.shap_values(df) computes per-feature contributions. Top 6 by absolute value extracted and sorted.
Step 7	Risk level assigned: failure_prob > 0.7 = high, > 0.4 = medium, else low.
Step 8	generate_explanation() in ai_explain.py: if low risk, returns static message. If medium/high, calls Gemini 2.5 Flash with structured prompt.
Step 9	insert_prediction() in database.py: stores id, IST timestamp, input_data, result, shap_values, ai_explanation, risk_level to SQLite.
Step 10	FastAPI returns PredictionResponse JSON: prediction_id, prediction_type, result, risk_level, shap_values, ai_explanation.
Step 11	Axios resolves. PredictionPage.jsx: setPrediction(response.data), setLoading(false). React re-renders.
Step 12	PredictionPanel shows failure/no-failure + confidence. ShapChart renders Recharts bar chart. AIExplanation splits text on 'ACTIONS' keyword and renders.

4.2 RUL Prediction Flow (Machines Page)

Step 1	Machines.jsx: User selects an engine unit from dropdown (MACHINE_UNITS list from Sensor.js). Clicks 'Run Prediction'.
Step 2	handleLoadHistory() calls predictRUL({ unit_id, sensor_readings }) via api.js. sensor_readings are pre-loaded dummy values from DUMMY_MACHINE_SENSORS[selectedUnit].
Step 3	FastAPI receives POST /predict/rul. Pydantic validates RULInput: unit_id (string) + sensor_readings (dict of floats).
Step 4	predict_rul(sensor_readings) in ml_utils.py: selects 14 sensor columns, applies MinMaxScaler (rul_scaler.pkl), calls rul_model.predict(). Returns RUL cycles as float.
Step 5	SHAP: rul_explainer.shap_values(df_scaled) on scaled data. Top 6 contributors extracted.
Step 6	Risk: RUL < 20 = high, < 60 = medium, else low. Gemini called if medium/high. Record saved to SQLite.
Step 7	Response returned. Machines.jsx: setPrediction(res.data). UI shows: RUL gauge circle (with risk border color), confidence, trend label (Healthy/Declining/Critical), SHAP chart, sensor table.

4.3 Dashboard Data Flow

Step 1	Dashboard.jsx mounts. useEffect calls useHistory({ limit: 200 }) from api.js — sends GET /history.
Step 2	history.py router queries SQLite: SELECT * FROM predictions ORDER BY created_at DESC LIMIT 200. Returns list of HistoryItem objects.
Step 3	Dashboard computes KPIs from returned array: uniqueMachines (Set of machine_unit), highRiskCount, avgRUL (filter rul predictions, average), todayCount (filter by today's date).
Step 4	Trend chart: predictions sorted by created_at, each assigned a RISK_SCORE (high=100, medium=50, low=10). Rolling 5-window average computed. Rendered as Recharts AreaChart.
Step 5	Pie chart: count of low/medium/high risk predictions. Rendered as Recharts PieChart with green/amber/red segments.
Step 6	Recent predictions table: last 8 entries sorted newest-first. Shows timestamp, type badge, machine unit, result, and risk status.

5. API Endpoints Reference

Method	Endpoint	Input (JSON)	Returns
POST	/predict/failure	air_temperature, process_temperature, rotational_speed, torque, tool_wear, machine_name (opt), machine_type (opt)	PredictionResponse with result, risk_level, shap_values, ai_explanation
POST	/predict/rul	unit_id (string), sensor_readings (dict of 14+ sensor: float pairs)	PredictionResponse with rul_cycles, risk_level, shap_values, ai_explanation
GET	/history	Query params: risk_level (opt), prediction_type (opt), limit (default 50)	List of HistoryItem with id, created_at, machine_unit, result, risk_level
GET	/	None	{ status: 'ok', service: 'predictive-maintenance-api' }
GET	/health	None	{ status: 'healthy' }

6. Data Schemas

6.1 Request Schemas (Pydantic — schemas.py)

FailureInput

```
machine_name      : str          (optional)
machine_type      : 'L'|'M'|'H'  (optional)
air_temperature   : float        (gt=0, in Kelvin)
process_temperature : float      (gt=0, in Kelvin)
rotational_speed  : float        (gt=0, in RPM)
torque            : float        (ge=0, in Nm)
tool_wear         : float        (ge=0, in minutes)
```

RULInput

```
unit_id          : str          (e.g. 'Unit_1')
sensor_readings  : dict[str, float] (keys: sensor2..sensor21, 14 values)
```

6.2 Response Schema (PredictionResponse)

```
prediction_id     : str          (UUID)
prediction_type    : 'failure' | 'rul'
result           : dict          (failure: {failure_predicted, confidence} | rul: {rul_cycles})
risk_level        : 'high' | 'medium' | 'low'
shap_values       : list[{feature: str, value: float}] (top 6, sorted by |value|)
ai_explanation      : str          (EXPLANATION\n...\n\nACTIONS\n- ...\n- ...)
```


7. Frontend State Management

React's built-in `useState` hook is used throughout — no external state library. Each page manages its own local state.

Page / Component	State Variable	Type	Purpose
PredictionPage	prediction	object null	Holds API response. null = no prediction yet. Drives conditional rendering of PredictionPanel, ShapChart, AIExplanation.
PredictionPage	loading	boolean	true while Axios call is in flight. Disables submit button, shows spinner.
PredictionPage	error	string null	Set to error message if API call fails. Cleared on each new submit.
Machines	selectedUnit	string	Currently selected engine unit (e.g. 'Unit_1'). Controls which sensor readings are sent.
Machines	prediction	object null	RUL API response. Drives RUL gauge, SHAP chart, AI explanation, sensor table.
Dashboard	predictions	array	Array of 200 most recent predictions fetched on mount. Source for all KPIs and charts.
Dashboard	trendData	array	Derived from predictions. Each entry: { cycle: number, avgRisk: number }. Fed to FailureTrendChart.

8. Key Component Interactions

8.1 How AIExplanation.jsx Parses Gemini Output

The Gemini API is instructed to return a strict format. The `parseExplanation()` function in `Machines.jsx` (and the equivalent in `AIExplanation.jsx`) handles parsing:

```
1. Split the full string on the keyword 'ACTIONS' into two parts
2. Part [0] = explanation block. Strip the word 'EXPLANATION' and .trim()
3. Part [1] = actions block. .trim() -> .split('\n') -> map(strip '-') -> filter(Boolean)
4. Result: { explanation: string, actions: string[] }
```

8.2 How SHAP Values Are Displayed

SHAP values come back as a list of { feature, value } objects sorted by absolute value (highest impact first). In the frontend:

- ShapChart.jsx renders a Recharts horizontal BarChart with layout='vertical'
- Each bar uses a Cell with fill='#f87171' (red) if value > 0 (increases risk), or '#22d3ee' (cyan) if value < 0 (reduces risk)
- Tooltip shows exact SHAP value on hover
- Legend: red = increases risk, cyan = reduces risk

8.3 Risk Colour Coding

Risk level drives colour throughout the UI — borders, badges, text, and gauge circles all change based on the returned `risk_level` string:

risk_level	Text Colour	Border	Background	Label
high	text-red-400	border-red-500	bg-red-500/10	HIGH RISK
medium	text-amber-400	border-amber-500	bg-amber-500/10	MEDIUM RISK
low	text-emerald-400	border-emerald-500	bg-emerald-500/10	LOW RISK

9. Database — How Data Is Stored and Retrieved

9.1 Table Schema (SQLite — predictions)

Column	Type	Description
id	TEXT (PK)	UUID generated per prediction (<code>uuid.uuid4()</code>)
created_at	TEXT	IST timestamp string — <code>datetime.now(IST).strftime('%Y-%m-%d %H:%M:%S')</code>
source	TEXT	'predict_input' (failure) or 'select_machine' (RUL)
machine_unit	TEXT	Machine name or engine unit ID. NULL if not provided.
machine_type	TEXT	'L', 'M', or 'H'. Only for failure predictions.
input_data	TEXT	JSON string of raw sensor values sent in the request.
prediction_type	TEXT	'failure' or 'rul'
result	TEXT	JSON string: {failure_predicted, confidence} or {rul_cycles}
shap_values	TEXT	JSON string: list of {feature, value} for top 6 sensors
ai_explanation	TEXT	Full Gemini explanation string (EXPLANATION + ACTIONS format)
risk_level	TEXT	'high', 'medium', or 'low'

9.2 How Data Flows Into the Database

- Every prediction — whether from `/predict/failure` or `/predict/rul` — calls `insert_prediction(record)` before returning the response.
- `result` and `shap_values` are Python dicts; they are serialised to JSON strings with `json.dumps()` before storage.
- When retrieved via `get_history()`, `_row_to_dict()` deserialises them back with `json.loads()` so the caller always gets Python dicts.
- The `get_history()` function supports optional filters: `risk_level`, `prediction_type`, and a `LIMIT` clause for pagination.

10. Google Gemini Integration Detail

The `generate_explanation()` function in `ai_explain.py` decides what to return based on risk level, then constructs and sends a structured prompt to Gemini 2.5 Flash.

10.1 Decision Logic

Risk Level	Action
low	Returns a pre-written static message (no API call). Saves latency and API cost.
medium / high	Calls <code>_call_gemini()</code> with prediction outcome, SHAP summary, and raw input readings.
API failure	Catches exception and returns <code>get_fallback_explanation()</code> with generic risk-level message.

10.2 What Is Sent to Gemini

- Prediction outcome string — e.g. 'Failure predicted: True, confidence 87.3%' or 'Remaining useful life: 42 cycles'
- SHAP summary — top feature names with signed values, e.g. 'tool_wear (+0.32), torque (+0.21), air_temp (-0.08)'
- Raw input readings dict — the exact sensor values the user submitted

10.3 Prompt Structure

Gemini is given a system-context paragraph and told to respond in a strict format only — no preamble, no markdown:

```
EXPLANATION
<3 sentences, plain language, no ML jargon>

ACTIONS
- <action 1>
- <action 2>
- <action 3>
```

This strict format is what allows the frontend to reliably split the response on the 'ACTIONS' keyword and render each section separately.

11. Machine Learning Models — Technical Detail

11.1 Failure Classification (XGBoost Classifier)

Parameter	Value
Dataset	AI4I 2020 Predictive Maintenance Dataset (10,000 rows)
Algorithm	XGBoost Classifier
n_estimators	100
max_depth	6
learning_rate	0.1
Features	air_temp, process_temp, rotational_speed, torque, tool_wear (5 features)
Train/Test split	80% / 20%, random_state=42
Accuracy	98.3%
Weighted F1 Score	0.98
Failure class Precision	0.79
Failure class Recall	0.61
Model file	failure_model.pkl

11.2 RUL Regression (XGBoost Regressor)

Parameter	Value
Dataset	NASA CMAPSS FD001 (train_FD001.txt — 20,631 rows)
Algorithm	XGBoost Regressor
n_estimators	100
max_depth	6
learning_rate	0.1
Features used	14 sensors: sensor2,3,4,7,8,9,11,12,13,14,15,17,20,21
Features dropped	sensor1,5,6,10,16,18,19 (near-constant, no predictive signal)
Preprocessing	MinMaxScaler fitted on training data. Scaler saved as rul_scaler.pkl
RUL label	max_cycle - current_cycle per engine_id
Train/Test split	80% / 20%, random_state=42
MAE	29.76 cycles
RMSE	41.54 cycles
Model file	rul_model.pkl + rul_scaler.pkl

12. Server Startup Sequence

When the FastAPI server starts (uvicorn main:app), the following happens in order before any request is served:

1. `lifespan()` context manager fires. `init_db()` is called — creates the predictions SQLite table if it does not exist.
2. `ml_utils.py` is imported. Module-level code runs: `failure_model.pkl`, `rul_model.pkl`, `rul_scaler.pkl` are loaded into memory with `pickle.load()`.
3. SHAP TreeExplainer objects are initialised for both models. This is the expensive step — done once so inference is fast per-request.
4. CORS middleware is registered to allow requests from the React dev server (configured via `settings.cors_origins` in `config.py` / `.env`).
5. Routers are registered: `predict.router` (prefix `/predict`) and `history.router`. Server is now ready to accept requests.